# Mikrotik: failover, load balancing

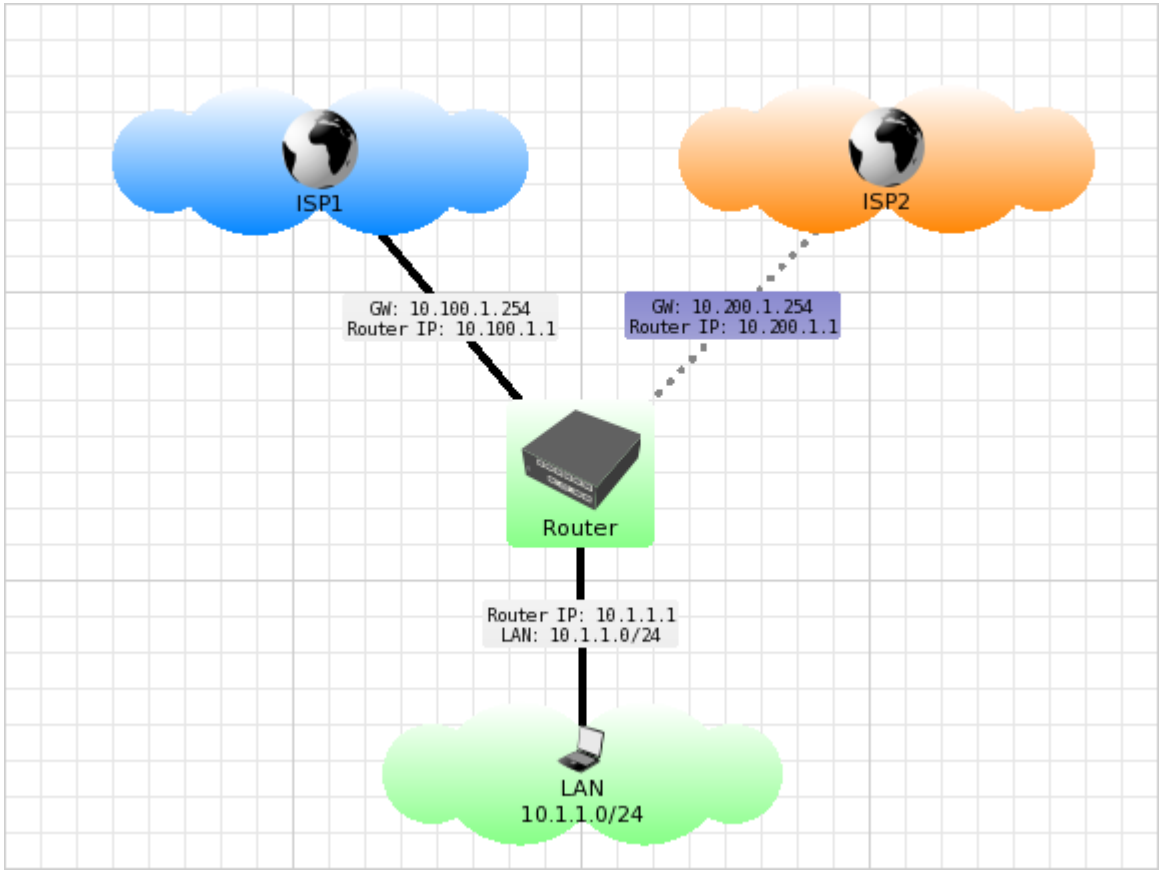
Когда у меня встала необходимость разобраться, как сделать failover или load balancing, имея два и более каналов в мир, я нашел множество статей и инструкций, в которых описывались рабочие конфигурации. Но почти нигде не нашел разъяснения, как все работает, и описания отличий разных вариантов. Хочу исправить эту несправедливость и собрать простейшие варианты построения failover и load balancing конфигураций в одной статье.

Итак, у нас есть роутер, который соединяет нашу локальную сеть и два канала в интернет (основной ISP1 и резервный ISP2).

Давайте рассмотрим что же мы можем сделать:

*Сразу предупрежу: несмотря на то, что в этой статье буду все описывать для mikrotik, не буду касаться темы скриптов*

## Failover




У нас появился резервный канал, в который можно направить трафик при отказе основного. Но как сделать, чтобы mikrotik понял, что канал упал?

**Простейшее резервирование каналов**

Простейший *failover* можно настроить, используя приоритет маршрута (distance у mikrotik/cisco, metric в linux/windows), а так же механизм проверки доступности шлюза — check-gateway.

В приведенной ниже конфигурации весь интернет трафик по умолчанию ходит через 10.100.1.254 (ISP1). Но как только адрес 10.100.1.254 станет недоступным (а маршрут через него неактивным) — трафик пойдет через 10.200.1.254 (ISP2).

**конфигурация: простейший failover**

```
# Настроим сети провайдеров:
/ip address add address=10.100.1.1/24 interface=ISP1
/ip address add address=10.200.1.1/24 interface=ISP2
# Настроим локальный интерфейс
/ip address add address=10.1.1.1/24 interface=LAN
# скроем за NAT все что выходит из локальной сети
/ip firewall nat add src-address=10.1.1.0/24 action=masquerade chain=srcnat
###Обеспечение резервирования каналов традиционным способом###
# укажем 2 default gateway с разными приоритетами
/ip route add dst-address=0.0.0.0/0 gateway=10.100.1.254 distance=1 check-
gateway=ping
/ip route add dst-address=0.0.0.0/0 gateway=10.200.1.254 distance=2 check-
gateway=ping
```

**check-gateway=ping для mikrotik обрабатывается так:**

Периодически (каждые 10 секунд) шлюз проверяется отсылкой на него ICMP пакета (ping). Потерянным пакет считается, если он не вернулся в течении 10 секунд. После двух потерянных пакетов шлюз считается недоступным. После получения ответа от шлюза он становится доступным и счетчик потерянных пакетов сбрасывается.

**Обеспечение failover с более глубоким анализом канала**

В прошлом примере все замечательно, кроме ситуации, когда шлюз провайдера виден и пингуется, но интернета за ним нет. Нам бы это здорово помогло, если бы можно было принимать решение о жизнеспособности провайдера, пингуя не сам шлюз, а что-то за ним.

Я знаю два варианта решения этой инженерной задачи. Первый и самый распространенный — использовать скрипты, но так как в этой статье мы скрипты не трогаем, остановимся подробнее на втором. Он подразумевает не совсем корректное использование параметра *scope*, но поможет нам прощупать канал провайдера глубже, чем до шлюза. Принцип прост:

Вместо традиционного указания default gateway=шлюз провайдера, мы скажем роутеру что default gateway это какой-то из *всегда_доступных_узлов* (например 8.8.8.8 или 8.8.4.4) и он в свою очередь доступен через шлюз провайдера.

**конфигурация: failover с более глубоким анализом канала**

```
# Настроим сети провайдеров:
/ip address add address=10.100.1.1/24 interface=ISP1
/ip address add address=10.200.1.1/24 interface=ISP2
# Настроим локальный интерфейс
/ip address add address=10.1.1.1/24 interface=LAN
# скроем за NAT все что выходит из локальной сети
/ip firewall nat add src-address=10.1.1.0/24 action=masquerade chain=srcnat
###Обеспечение failover с более глубоким анализом канала###
#с помощью параметра scope укажем рекурсивные пути к узлам 8.8.8.8 и 8.8.4.4
/ip route add dst-address=8.8.8.8 gateway=10.100.1.254 scope=10
/ip route add dst-address=8.8.4.4 gateway=10.200.1.254 scope=10
```

```
# укажем 2 default gateway через узлы путь к которым указан рекурсивно
/ip route add dst-address=0.0.0.0/0 gateway=8.8.8.8 distance=1 check-
gateway=ping
/ip route add dst-address=0.0.0.0/0 gateway=8.8.4.4 distance=2 check-
gateway=ping
```

Теперь разберем, что происходит чуть подробнее:

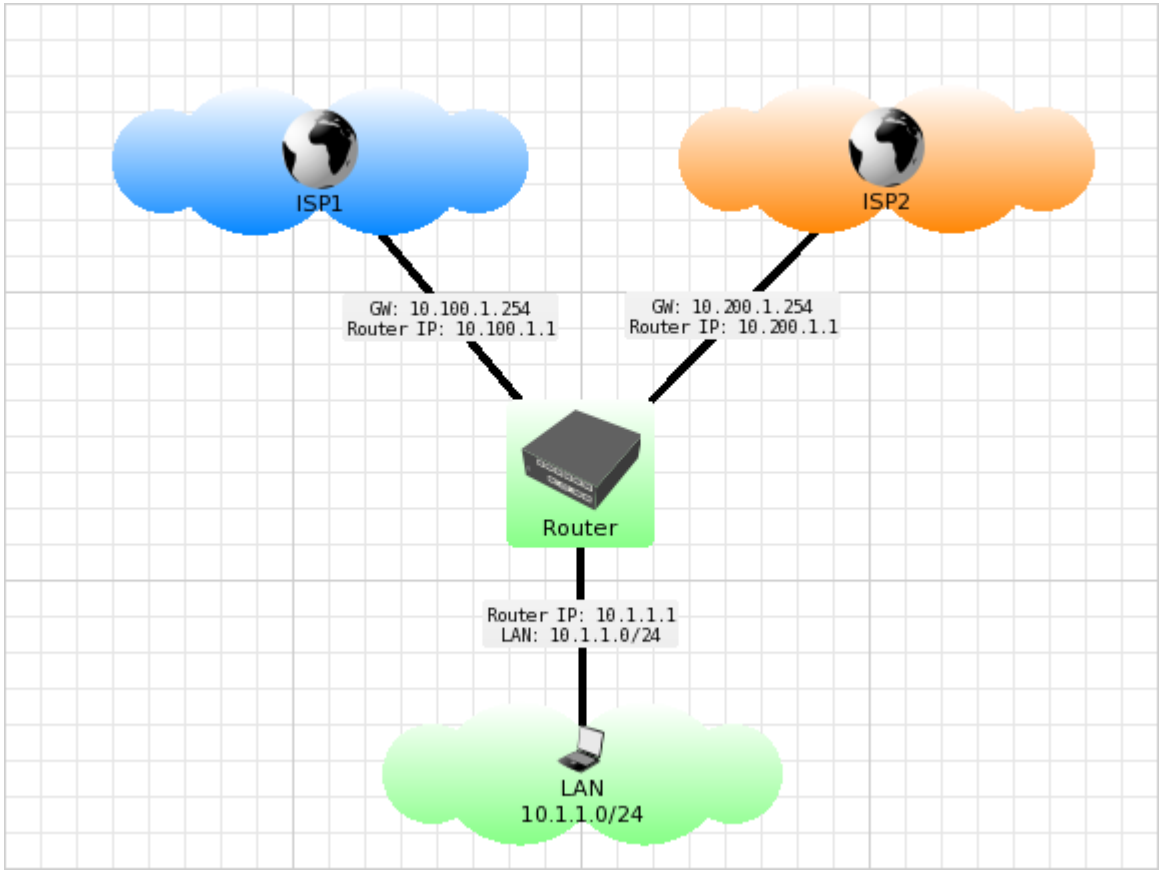
Хитрость в том, что шлюз провайдера не знает о том, что 8.8.8.8 или 8.8.4.4 — это роутер и направит трафик по обычному пути.

Наш mikrotik считает, что по умолчанию весь интернет трафик нужно отправлять на 8.8.8.8, который напрямую не виден, но через 10.100.1.254 доступен. А если пинг на 8.8.8.8 пропадает (напомню что путь к нему у нас жестко указан через шлюз от ISP1) то mikrotik начнет слать весь интернет трафик на 8.8.4.4, а точнее на рекурсивно определенный 10.200.1.254 (ISP2).

*Но у меня пару раз возникала ситуация, когда интернет через шлюз провайдера работает, а вот конкретный узел или сеть нет. В таких случаях вышеописанный метод не очень-то помогает и для обеспечения бесперебойной работы мне приходилось проверять доступность узла уже с помощью скриптов. Кстати, если кто знает решение файловера на один внешний хост без применения скриптов и протоколов динамической маршрутизации — поделитесь рецептом.*

# Load Balancing

Теперь давайте рассмотрим другую схему:




В ней второй второй канал уже не резервный, а равноценный. Почему бы не

использовать оба канала одновременно, повысив таким образом пропускную способность?

## Начинаем настраивать Load Balancing

*Первое правило Load Balancing* — следить за соединениями: на пришедшее извне соединение отвечать с того же адреса, на который оно пришло. Для исходящих соединений — отправлять пакеты только через тот адрес, с которым установилось соединение.

*Второе, что тоже важно понимать* — нужно разделять понятия входящего и исходящего трафика. Дело в том, что для исходящего трафика роутер может решать, по какому пути он пойдет, а входящий трафик для него как «трафик Шредингера». Пока его нет, наш mikrotik не знает, через какой интерфейс он придет, а когда пришел — менять интерфейс уже поздно.

*Третье* — балансировка каналов не является резервированием. Это две отдельные функции.

**Кстати почему при разделе трафика мы оперируем соединениями, а не пакетами?**

Почитайте, как работает TCP протокол. Если коротко — задача TCP протокола не только швырнуть пакетом в получателя, но и проконтролировать, как тот его принял. Делается это с помощью установки соединения, в рамках которого, собственно, и передаются пакеты данных — вместе со служебной информацией. Если оперировать пакетами и забыть про соединения, то возможны ситуации, когда удаленный хост, установив соединение с одним адресом, просто отбросит часть пакетов пришедшую со второго — «неправильного» адреса.

### Готовимся принять «трафик Шредингера»

Итак, при любом варианте балансировки канала нам нужно сначала подготовиться к принятию входящего трафика и научить mikrotik отвечать на входящие соединения в тот же канал, из которого они пришли. С помощью маркировки соединений и последующей маркировки маршрутов для них мы, по сути, делаем несколько таблиц маршрутизации, отдельных для каждого внешнего адреса.

**начальная конфигурация для load balancing с двумя внешними IP адресами**

```
# Настроим сети провайдеров:
/ip address add address=10.100.1.1/24 interface=ISP1
/ip address add address=10.200.1.1/24 interface=ISP2
# Настроим локальный интерфейс
/ip address add address=10.1.1.1/24 interface=LAN
# скроем за NAT все что выходит из локальной сети
/ip firewall nat add src-address=10.1.1.0/24 action=masquerade chain=srcnat
#Пометим каждое соединение пришедшее снаружи и адресованное нашему роутеру:
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP1
new-connection-mark=cin_ISP1
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP2
new-connection-mark=cin_ISP2
#что бы отвечать через те же интерфейсы, откуда пришли запросы, поставим
соответствующую роутинг-марку на каждое соединение.
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP1 new-routing-mark=rout_ISP1 passthrough=no
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP2 new-routing-mark=rout_ISP2 passthrough=no
#добавим default gateway в каждую из промаркированных таблиц маршрутизации:
/ip route add distance=1 gateway=10.100.1.254 routing-mark=rout_ISP1 check-
gateway=ping
```

```
/ip route add distance=1 gateway=10.200.1.254 routing-mark=rout_ISP2 check-
gateway=ping
```

Таким образом mikrotik будет проводить каждый пакет помеченного соединения по соответствующей таблице маршрутизации и внешние адреса (10.100.1.1, 10.200.1.1) будут доступны извне без путаницы в каналах и маршрутах.

## Делим исходящий трафик

Для распределения исходящего трафика по интерфейсам, нам всего-то нужно повесить соответствующую марку маршрута на соединение. Сложность в том, что нужно решить на какое соединение вешать метку ISP1, а на какое ISP2.

Существует несколько вариантов разделения соединений по группам:

### 1) Делим исходящий трафик, прикручивая марку намертво

Правила, балансирующие трафик мы можем прописать жестко:
Например мы хотим настроить важные для нас протоколы HTTP(80 port), HTTPS (443 port), POP (110 port), SMTP (25 port) через ISP1, а весь остальной трафик пустить через второго провайдера:

**конфигурация с балансировкой канала по жестко прописанным правилам**

```
# Настроим сети провайдеров:
/ip address add address=10.100.1.1/24 interface=ISP1
/ip address add address=10.200.1.1/24 interface=ISP2
# Настроим локальный интерфейс
/ip add address=10.1.1.1/24 interface=LAN
# скроем за NAT все что выходит из локальной сети
/ip firewall nat add src-address=10.1.1.0/24 action=masquerade chain=srcnat
#Пометим каждое соединение пришедшее снаружи и адресованное нашему роутеру:
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP1
new-connection-mark=cin_ISP1
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP2
new-connection-mark=cin_ISP2
#что бы отвечать через те же интерфейсы, откуда пришли запросы, поставим
соответствующую роутинг-марку на каждое соединение.
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP1 new-routing-mark=rout_ISP1 passthrough=no
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP2 new-routing-mark=rout_ISP2 passthrough=no
#добавим default gateway в каждую из промаркированных таблиц маршрутизации:
/ip route add distance=1 gateway=10.100.1.254 routing-mark=rout_ISP1 check-
gateway=ping
/ip route add distance=1 gateway=10.200.1.254 routing-mark=rout_ISP2 check-
gateway=ping
#failover через второго провайдера для каждого из шлюзов
/ip route add distance=2 gateway=10.200.1.254 routing-mark=rout_ISP1
/ip route add distance=2 gateway=10.100.1.254 routing-mark=rout_ISP2
# отправим трафик идущий к портам 80,443,110,25 на ISP1
/ip firewall mangle add chain=prerouting action=mark-routing new-routing-
mark="lan_out_ISP1" passthrough=no dst-port=80,443,110,25 protocol=tcp
#а весь остальной трафик на ISP2:
/ip firewall add chain=prerouting action=mark-routing new-routing-
mark="lan_out_ISP2" passthrough=no
#добавим default gateway в каждую из промаркированных для LAN трафика таблиц
маршрутизации:
/ip route add distance=1 gateway=10.100.1.254 routing-mark=lan_out_ISP1 check-
```

```
gateway=ping
/ip route add distance=1 gateway=10.200.1.254 routing-mark=lan_out_ISP2 check-
gateway=ping
#failover через второго провайдера для каждого из шлюзов
/ip route add distance=2 gateway=10.200.1.254 routing-mark=lan_out_ISP1
/ip route add distance=2 gateway=10.100.1.254 routing-mark=lan_out_ISP2
```

**2) Делим исходящий трафик, выбирая каждое N-ое соединение**

Можем делить соединения по-порядку. Первые налево, вторые — направо. Все просто.

**конфигурация с балансировкой канала по N-ному соединению:**

```
# Настроим сети провайдеров:
/ip address add address=10.100.1.1/24 interface=ISP1
/ip address add address=10.200.1.1/24 interface=ISP2
# Настроим локальный интерфейс
/ip address add address=10.1.1.1/24 interface=LAN
# скроем за NAT все что выходит из локальной сети
/ip firewall nat add src-address=10.1.1.0/24 action=masquerade chain=srcnat
#Пометим каждое соединение пришедшее снаружи и адресованное нашему роутеру:
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP1
new-connection-mark=cin_ISP1
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP2
new-connection-mark=cin_ISP2
#что бы отвечать через те же интерфейсы, откуда пришли запросы, поставим
соответствующую роутинг-марку на каждое соединение.
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP1 new-routing-mark=rout_ISP1 passthrough=no
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP2 new-routing-mark=rout_ISP2 passthrough=no
#добавим default gateway в каждую из промаркированных для внешних адресов таблиц
маршрутизации:
/ip route add distance=1 gateway=10.100.1.254 routing-mark=rout_ISP1 check-
gateway=ping
/ip route add distance=1 gateway=10.200.1.254 routing-mark=rout_ISP2 check-
gateway=ping
#из каждых двух соединений первое пометим на отсылку через ISP1
/ip firewall mangle add src-address=10.1.1.0/24 action=mark-routing
chain=prerouting new-routing-mark=lan_out_ISP1 nth=2,1
#а второе на отсылку через ISP2
/ip firewall mangle add src-address=10.1.1.0/24 action=mark-routing
chain=prerouting new-routing-mark=lan_out_ISP2 nth=2,2
#добавим default gateway в каждую из промаркированных для LAN трафика таблиц
маршрутизации:
/ip route add distance=1 gateway=10.100.1.254 routing-mark=lan_out_ISP1 check-
gateway=ping
/ip route add distance=1 gateway=10.200.1.254 routing-mark=lan_out_ISP2 check-
gateway=ping
#failover через второго провайдера для каждого из шлюзов
/ip route add distance=2 gateway=10.200.1.254 routing-mark=lan_out_ISP1
/ip route add distance=2 gateway=10.100.1.254 routing-mark=lan_out_ISP2
```

**3) Делим исходящий трафик, используя PCC (per connection classifier)**

PCC подходит к дележу трафика чуть сложнее. Он разделяет трафик по группам, основываясь на данных TCP заголовка (src-address, dst-address, src-port, dst-port).

**конфигурация с балансировкой канала по PPC:**

```
# Настроим сети провайдеров:
```

```
/ip address add address=10.100.1.1/24 interface=ISP1
/ip address add address=10.200.1.1/24 interface=ISP2
# Настроим локальный интерфейс
/ip address add address=10.1.1.1/24 interface=LAN
# скроем за NAT все что выходит из локальной сети
/ip firewall nat add src-address=10.1.1.0/24 action=masquerade chain=srcnat
#Пометим каждое соединение пришедшее снаружи и адресованное нашему роутеру:
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP1
new-connection-mark=cin_ISP1
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP2
new-connection-mark=cin_ISP2
#что бы отвечать через те же интерфейсы, откуда пришли запросы, поставим
соответствующую роутинг-марку на каждое соединение.
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP1 new-routing-mark=rout_ISP1 passthrough=no
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP2 new-routing-mark=rout_ISP2 passthrough=no
#добавим default gateway в каждую из промаркированных таблиц маршрутизации:
/ip route add distance=1 gateway=10.100.1.254 routing-mark=rout_ISP1 check-
gateway=ping
/ip route add distance=1 gateway=10.200.1.254 routing-mark=rout_ISP2 check-
gateway=ping
#failover через второго провайдера для каждого из шлюзов
/ip route add distance=2 gateway=10.200.1.254 routing-mark=rout_ISP1
/ip route add distance=2 gateway=10.100.1.254 routing-mark=rout_ISP2
#используя PPC разделим трафик на две группы по исх. адресу и порту
/ip firewall mangle add src-address=10.1.1.0/24 action=mark-routing
chain=prerouting new-routing-mark=lan_out_ISP1 per-connection-classifier=src-
address-and-port:2/0
/ip firewall mangle add src-address=10.1.1.0/24 action=mark-routing
chain=prerouting new-routing-mark=lan_out_ISP2 per-connection-classifier=src-
address-and-port:2/1
#добавим default gateway в каждую из промаркированных для LAN трафика таблиц
маршрутизации:
/ip route add distance=1 gateway=10.100.1.254 routing-mark=lan_out_ISP1 check-
gateway=ping
/ip route add distance=1 gateway=10.200.1.254 routing-mark=lan_out_ISP2 check-
gateway=ping
#failover через второго провайдера для каждого из шлюзов
/ip route add distance=2 gateway=10.200.1.254 routing-mark=lan_out_ISP1
/ip route add distance=2 gateway=10.100.1.254 routing-mark=lan_out_ISP2
```

**Делим исходящий трафик, используя ECMP (equal cost multipath routing)**

На мой взгляд самый простой и вкусный вариант разделения траффика:

**конфигурация с балансировкой канала по ECMP**

```
# Настроим сети провайдеров:
/ip address add address=10.100.1.1/24 interface=ISP1
/ip address add address=10.200.1.1/24 interface=ISP2
# Настроим локальный интерфейс
/ip address add address=10.1.1.1/24 interface=LAN
# скроем за NAT все что выходит из локальной сети
/ip firewall nat add src-address=10.1.1.0/24 action=masquerade chain=srcnat
#Пометим каждое соединение пришедшее снаружи и адресованное нашему роутеру:
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP1
new-connection-mark=cin_ISP1
/ip firewall mangle add action=mark-connection chain=input in-interface=ISP2
new-connection-mark=cin_ISP2
#что бы отвечать через те же интерфейсы, откуда пришли запросы, поставим
соответствующую роутинг-марку на каждое соединение.
```

```
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP1 new-routing-mark=rout_ISP1 passthrough=no
/ip firewall mangle add action=mark-routing chain=output connection-
mark=cin_ISP2 new-routing-mark=rout_ISP2 passthrough=no
#добавим default gateway в каждую из промаркированных таблиц маршрутизации:
/ip route add distance=1 gateway=10.100.1.254 routing-mark=rout_ISP1 check-
gateway=ping
/ip route add distance=1 gateway=10.200.1.254 routing-mark=rout_ISP2 check-
gateway=ping
#промаркируем весь траффик из локальной сети
/ip firewall mangle add src-address=10.1.1.0/24 action=mark-routing
chain=prerouting new-routing-mark=mixed
#используем ECMP для балансировки траффика из локальной сети
/ip route add dst-address=0.0.0.0/0 gateway=10.100.1.254,10.200.1.254 routing-
mark=mixed
```

Mikrotik сам поделит трафик по шлюзам, используя round-robin алгоритм.

Кстати, в версии 6.X RouterOS mikrotik поломал check-gateway в ECMP, так что использовать конструкцию
*/ip route add gateway=10.100.1.254,10.200.1.254 check-gateway=ping* можно и логично, но абсолютно бесполезно.

Для пометки неживых маршрутов в ECMP нужно создать дополнительные маршруты, которые используют каждый из gateway по-отдельности. С включенным check-gateway разумеется. Помечая маршрут неактивным, mikrotik делает это для всех.

**И последнее немаловажное замечание про скорость каналов**

Возьмем 2 неравнозначных канала, например, 100 мбит и 50 мбит. Сбалансируем их через Nth, PCC или ECMP. Какую суммарную пропускную способность получим?

На самом деле — где-то около 100 мбит (самый слабый канал X раз).
Происходит это потому, что mikrotik понятия не имеет о пропускной способности каналов, он ее не измеряет. Он просто делит трафик на относительно равные группы.

Побороть этот нюанс можно правильно проектируя группы исходящего трафика, с учетом пропускной способности каналов.

**Например**
в ECMP это можно сделать указав более скоростной шлюз несколько раз, тем самым повышая частоту его использования.

```
/ip route add dst-address=0.0.0.0/0
gateway=10.100.1.254,10.100.1.254,10.200.1.254
```

В PCC тоже можно сделать неравные группы:

```
/ip firewall mangle add src-address=10.1.1.0/24 action=mark-routing
chain=prerouting new-routing-mark=lan_out_ISP1 per-connection-classifier=src-
address-and-port:3/0
/ip firewall mangle add src-address=10.1.1.0/24 action=mark-routing
chain=prerouting new-routing-mark=lan_out_ISP1 per-connection-classifier=src-
address-and-port:3/1
/ip firewall mangle add src-address=10.1.1.0/24 action=mark-routing
chain=prerouting new-routing-mark=lan_out_ISP2 per-connection-classifier=src-
address-and-port:3/2
```